**Relatório 9 - Prática: Git e Github para Iniciantes (III)**

Gustavo Nunes da Silva

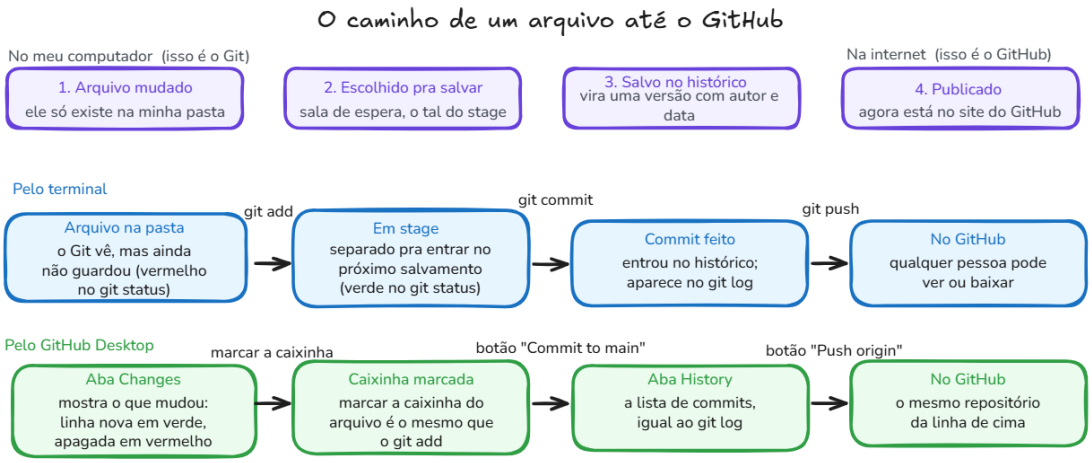
## 1. Introdução

Nesse card eu aprendi a usar o Git e o GitHub pra guardar o histórico do meu código. Eu segui o curso do Willian Justen na Udemy e assisti um vídeo sobre o GitHub Desktop. Depois eu refiz tudo no meu computador. Criei um repositório do zero e também clonei o repositório que o professor usa nas aulas.

## 2. Desenvolvimento

A primeira coisa que eu aprendi é que Git e GitHub não são a mesma coisa. O Git é um programa que fica no meu computador e guarda cada versão dos meus arquivos. O GitHub é um site que recebe esse projeto e deixa qualquer pessoa ver ou baixar. Dá pra usar o Git sozinho e nunca mandar nada pra internet. O contrário não dá, porque o GitHub só guarda o que o Git preparou antes.

Figura 1



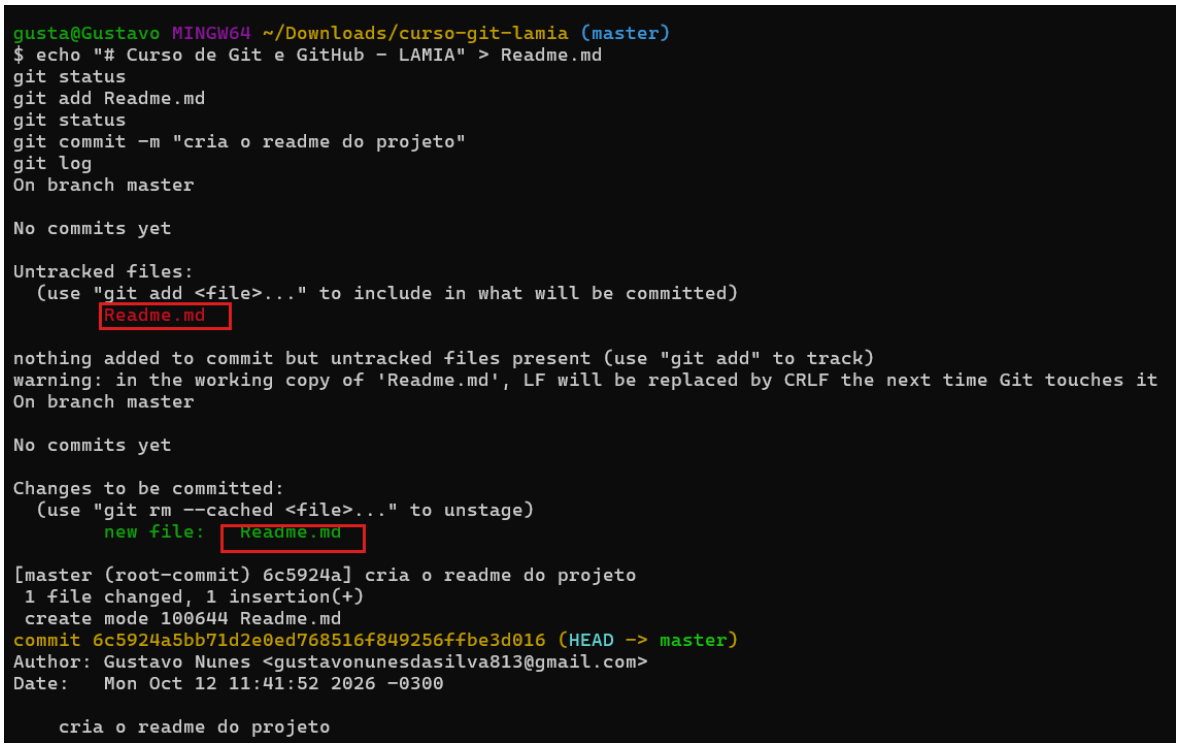

Fonte – Autoria própria (elaborado no Excalidraw).

O curso tem seis seções e trinta aulas. Ele começa falando o que é controle de versão e conta que o Git foi criado pelo Linus Torvalds em 2005 pro Linux. Depois vem a configuração, onde eu coloquei meu nome e meu e-mail com o git config. Isso é importante porque esses dados ficam gravados em todo commit que eu faço.

A parte que eu mais gostei foi o ciclo de vida dos arquivos. Um arquivo novo começa como untracked, que é quando o Git vê ele mas ainda não guarda. Com

o git add ele vai pra uma sala de espera chamada stage. Só com o git commit ele entra de verdade no histórico. Eu rodei o git status antes e depois do git add pra ver isso acontecer. A diferença aparece até na cor, porque o arquivo sai do vermelho e vai pro verde.

Figura 2

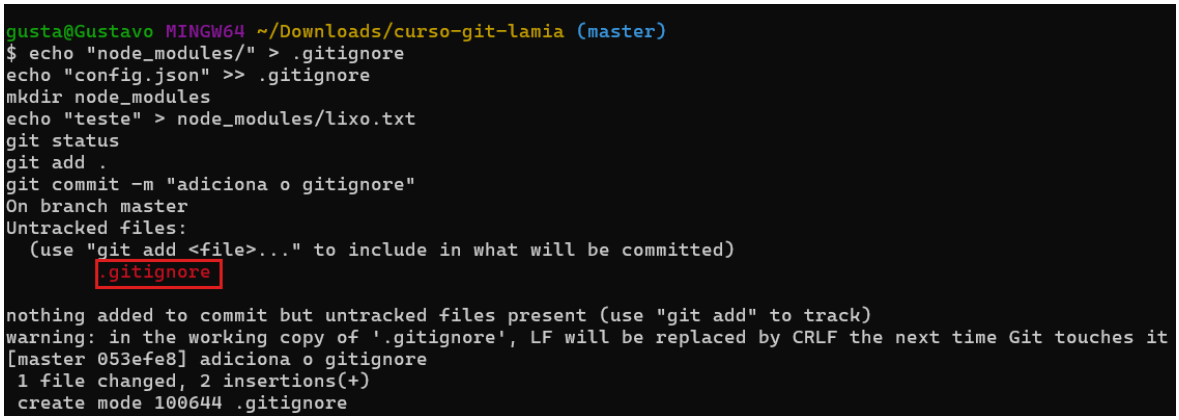



Fonte – Autoria própria (print do Git Bash).

Outro comando que me ajudou foi o git log. Ele lista todos os commits com o autor, a data e um código de identificação. Esse código é um monte de letra e número que o Git cria pra cada commit, e é por ele que eu consigo voltar numa versão antiga. O git diff faz algo parecido, mas mostra linha por linha o que mudou desde o último salvamento.

Depois eu criei o arquivo .gitignore. Ele serve pra dizer ao Git o que ele deve ignorar. Eu coloquei a pasta node_modules e o arquivo config.json dentro dele. Criei a pasta de propósito e rodei o git status de novo. Só o .gitignore apareceu na lista, e a pasta foi tratada como se não existisse. Isso evita subir arquivo pesado que não serve pra nada e também protege senha, porque é nesse tipo de arquivo que a gente guarda chave de acesso.

Figura 3

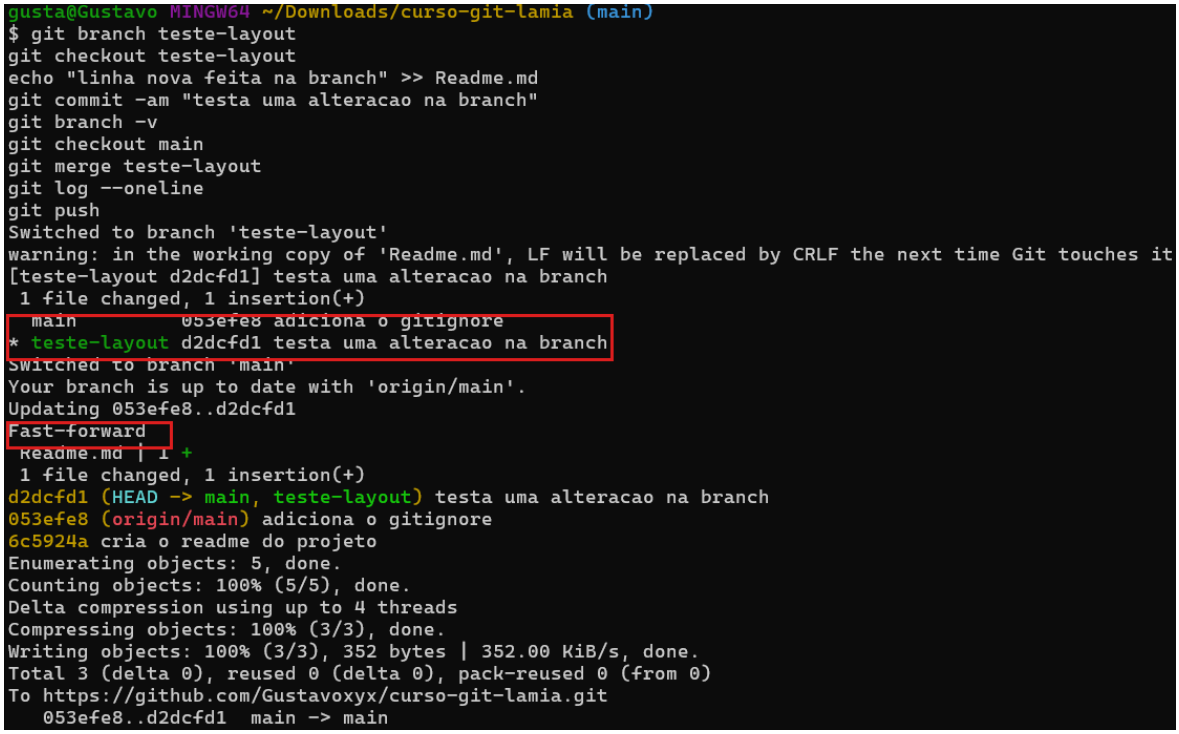



Fonte – Autoria própria (print do Git Bash).

Com o repositório funcionando no computador eu fui pros repositórios remotos. Criei um repositório vazio no GitHub, liguei ele ao meu projeto com o git remote add origin e mandei tudo com o git push. Aqui eu achei uma diferença entre o curso e o GitHub de hoje. O Git ainda cria a primeira branch com o nome master, que é o nome que aparece nas aulas, mas o GitHub usa main. Eu renomeei a minha com o git branch -M main antes de enviar, senão as duas ficariam com nomes diferentes.

A parte de branch foi a mais longa do curso e também a mais útil. Uma branch é uma linha de trabalho separada. Nela eu posso testar uma ideia sem estragar a versão boa do projeto. Eu criei a branch teste-layout, escrevi uma linha nova no arquivo e fiz um commit só dela. Quando eu voltei pra main a linha tinha sumido. Ela só voltou depois do git merge. O Git avisou que foi um Fast-forward, que é quando a main não mudou enquanto eu trabalhava na outra branch. Aí ele só puxa o ponteiro pra frente em vez de criar um commit de junção.

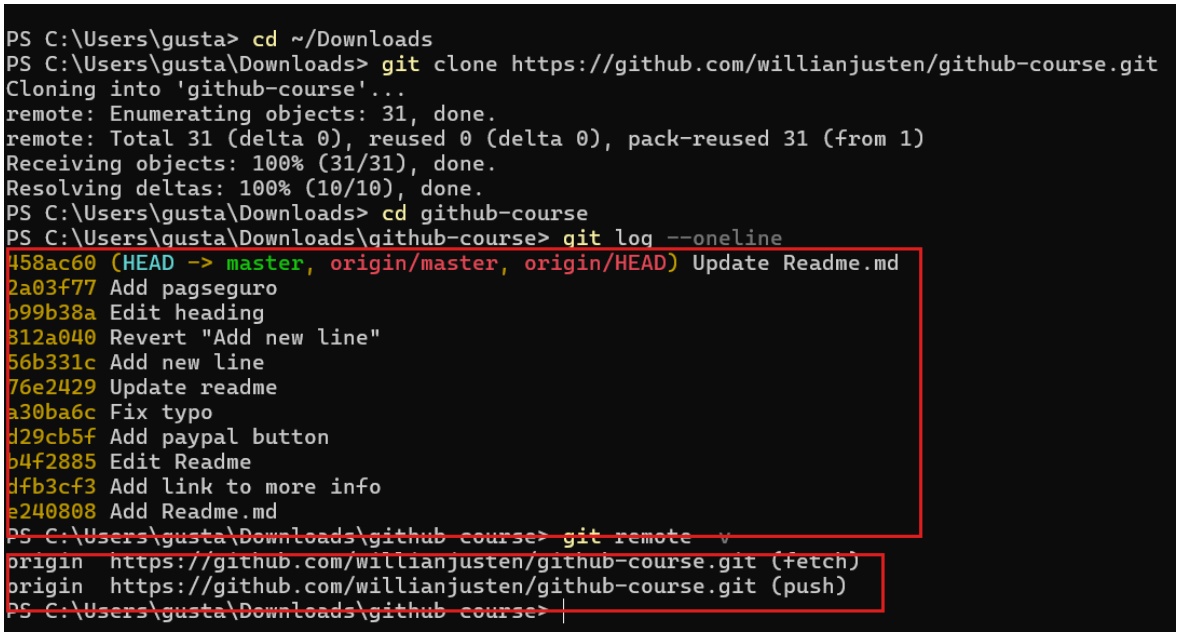
Figura 4




Fonte – Autoria própria (print do Git Bash).

A segunda parte da prática foi no repositório que o professor usa no curso. Eu rodei o git clone no endereço dele e em poucos segundos os onze commits apareceram no meu computador. Essa é a diferença entre o clone e o git init. O init começa um projeto vazio e o clone traz o histórico inteiro junto, com todos os commits e o endereço de origem já configurado.

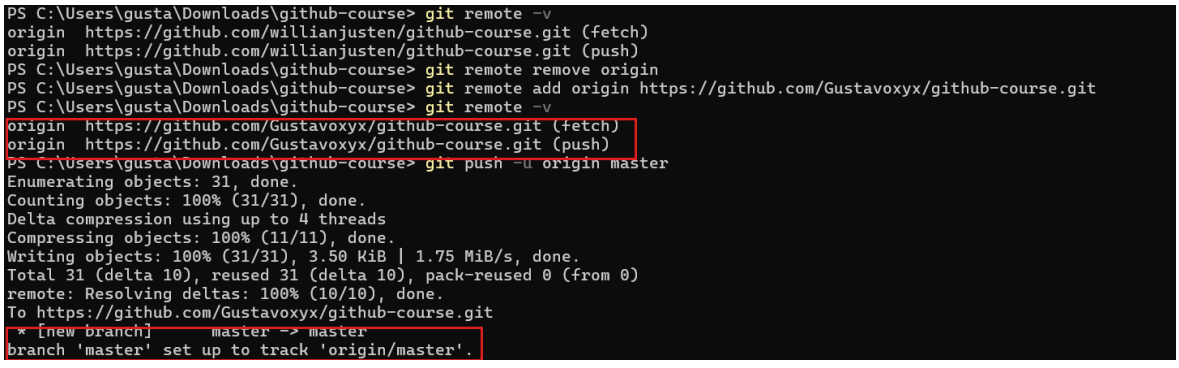
Figura 5




Fonte – Autoria própria (print do PowerShell).

Como eu queria esse projeto dentro da minha conta, eu troquei o destino. Tirei o origin que apontava pro repositório do professor, coloquei o endereço do meu e rodei o git push. O GitHub criou a branch master lá e os onze commits foram parar no meu perfil. Cada um ainda tem o nome do autor original, porque o Git guarda isso junto com o commit e não troca por quem fez o envio.

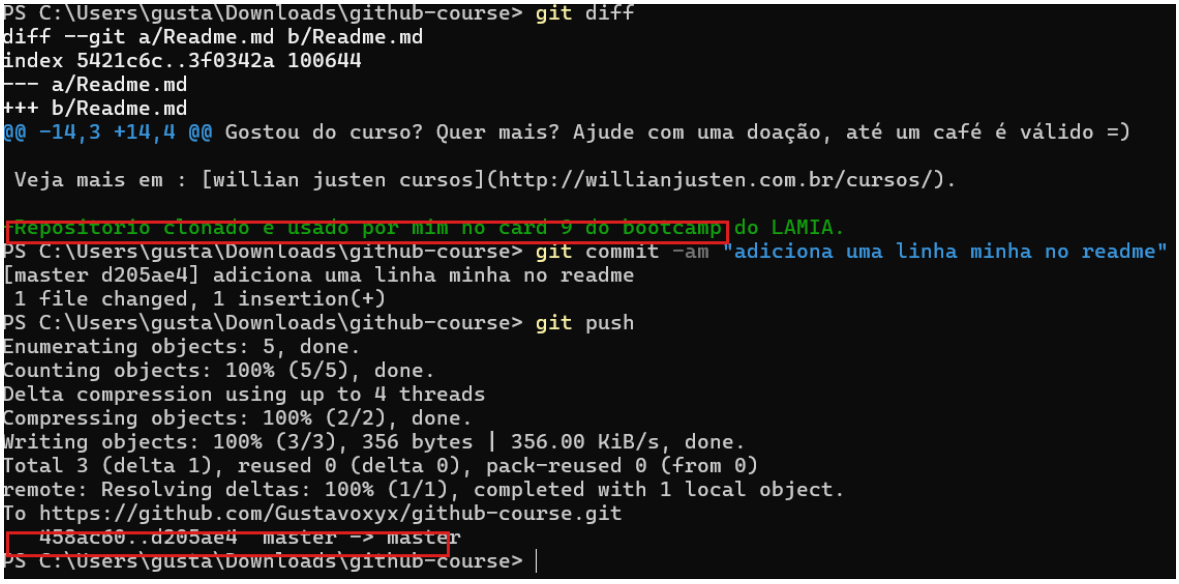
Figura 6




Fonte – Autoria própria (print do PowerShell).

Pra fechar eu escrevi uma linha minha no final do Readme e conferi com o git diff antes de salvar. O Git mostrou a linha nova com um sinal de mais na frente e pintou ela de verde.

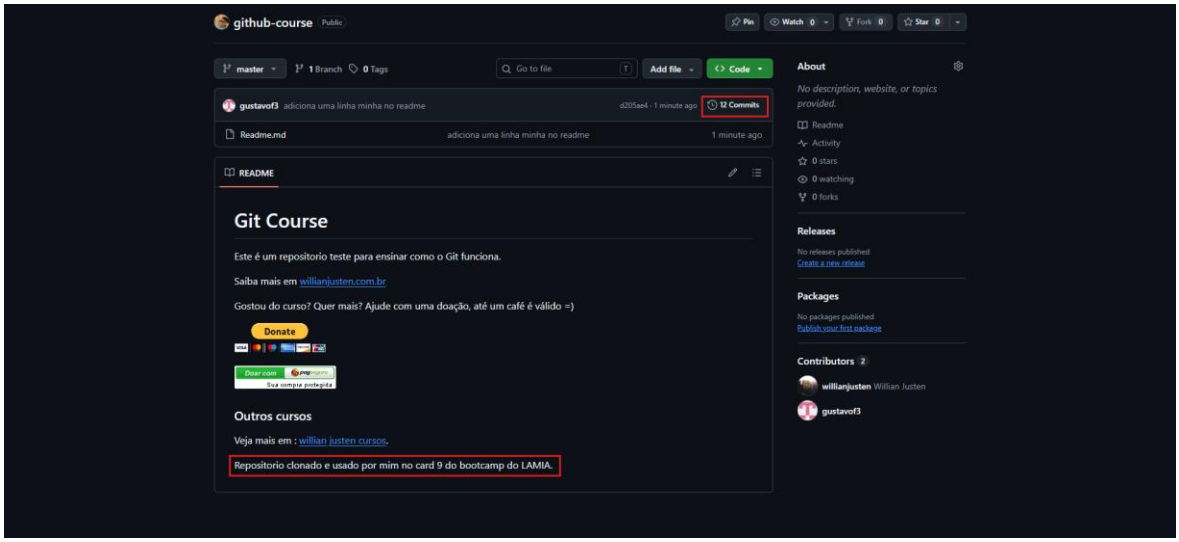
Figura 7




Fonte – Autoria própria (print do PowerShell).

Depois do push o repositório ficou com doze commits, sendo o último meu e os outros onze do professor. Abrindo a página no navegador dá pra ver o texto dele e o meu no mesmo arquivo.

Figura 8




Fonte – Autoria própria (print do navegador).

Fora o curso eu assisti um vídeo que ensina o GitHub Desktop. É um programa com telas e botões no lugar dos comandos. Ele faz o mesmo trabalho do terminal, só com outro nome. A aba Changes mostra o que eu mudei desde o

último salvamento, com as linhas apagadas em vermelho e as novas em verde, igual ao git diff. Marcar a caixinha do arquivo é o git add. O botão de commit é o git commit e o botão de push é o git push. A aba History mostra a lista de commits igual ao git log.

O vídeo fala de um assunto que o curso só cita de passagem, que é o cuidado com o token do bot. Se eu subir um arquivo com o token num repositório público qualquer pessoa pode pegar e usar o meu bot. A saída é guardar o token num arquivo separado e colocar esse arquivo no .gitignore. Assim ele fica só no meu computador. O vídeo também mostra o botão Discard changes, que apaga a alteração e devolve o arquivo como ele estava antes.

No fim eu gostei mais do terminal do que do programa. O programa é mais fácil de olhar e bom pra quem está começando, mas ele esconde o que está acontecendo. Digitando os comandos eu entendi de onde vem cada etapa, e isso me ajudou a sacar o que o botão faz por baixo.

**3. Conclusão**

Esse card mudou o jeito que eu vejo salvar código, porque antes eu só copiava pasta pra não perder o que estava funcionando. O que mais ficou pra mim foi o ciclo de vida dos arquivos, que explica por que o commit precisa de dois passos e não de um só. Também achei legal ver que o clone traz o histórico inteiro e não só os arquivos, e que cada commit guarda quem escreveu ele mesmo depois de ir pra outra conta. As branches foram o que mais me deu segurança pra testar coisa nova, porque o projeto principal fica intacto. E o vídeo do GitHub Desktop mostrou que o botão e o comando fazem a mesma coisa.

**4. Referências**

JUSTEN, Willian. Git e Github para iniciantes. Udemy, 2020. Disponível em: https://www.udemy.com/course/git-e-github-para-iniciantes/. Acesso em: 12 out. 2026.

JUSTEN, Willian. github-course. GitHub. Disponível em: https://github.com/willianjusten/github-course. Acesso em: 12 out. 2026.

RICARDO, Pedro. Como usar o Github Desktop. YouTube. Disponível em: https://www.youtube.com/watch?v=Fj3gtbaF8WA. Acesso em: 12 out. 2026.